

VIPulse AI - High Value Engineering Challenges & Fixes

1. MongoDB + Beanie + PyMongo Dependency Conflict

Error:
ImportError: cannot import name '_QUERY_OPTIONS' from pymongo.cursor

Root Cause:
Incompatible versions of pymongo, motor, and beanie caused backend startup failure.

How I Fixed It:
I manually checked compatibility and pinned stable versions.

Exact Changes:
motor==3.4.0
beanie==1.25.0
pymongo==4.6.3

Why This Matters:
This stabilized async MongoDB communication and prevented crashes.

2. JWT Authentication Failure (401 Unauthorized)

Error:
GET /api/v1/dashboard/stats -> 401 Unauthorized

Root Cause:
Frontend was sending stale JWT tokens and Authorization headers were missing.

How I Fixed It:
I verified Axios interceptors and token storage.

Exact Code Change:
axios.interceptors.request.use((config) => {
 const token = localStorage.getItem("access_token");
 if (token) {
 config.headers.Authorization = `Bearer \${token}`;
 }
 return config;
});

Why This Matters:
Restored secure authenticated API access.

3. Frontend-Backend Response Contract Mismatch

Error:
Frontend showed 'No Tickets Found' despite backend returning data.

Root Cause:
Frontend expected response.data.items
Backend returned response.data.data

How I Fixed It:
Aligned frontend parsing logic.

Exact Code Change:
const tickets = response.data.data;

Why This Matters:
Restored ticket rendering and API consistency.

4. Multi-User Data Leakage Problem

Error:
All users saw same analytics and tickets.

Root Cause:
Backend queries were not scoped to authenticated users.

How I Fixed It:
Added user-based filtering.

Exact Code Change:

```
tickets = await Ticket.find({  
  "department": current_user.department  
})
```

Why This Matters:
Enforced RBAC and user data isolation.

5. security.py Syntax Failure During Authentication Refactor

Error:
SyntaxError: invalid syntax in security.py

Root Cause:
Two functions accidentally merged during code generation.

Broken:

```
return await loop.run_in_executor(...)def verify_password(...)
```

How I Fixed It:
Separated sync and async password verification logic.

Exact Code Change:

```
async def verify_password_async(plain, hashed):  
    loop = asyncio.get_running_loop()  
    return await loop.run_in_executor(  
        _hash_executor,  
        verify_password,  
        plain,  
        hashed  
    )
```

```
def verify_password(plain, hashed):  
    return pwd_context.verify(plain, hashed)
```

Why This Matters:
Restored authentication reliability.